

SKILL WEEK11

TASK1: This task is about creating a small program with **separate modules**, connecting them, compiling them together, testing the complete flow, and handling errors.

```
#ifndef CALCULATOR_H
#define CALCULATOR_H

int add(int a, int b);
int subtract(int a, int b);

#endif
```

```
#include "calculator.h"

int add(int a, int b)
{
    return a + b;
}

int subtract(int a, int b)
{
    return a - b;
}
```

```
#include <stdio.h>
#include "calculator.h"

int main()
{
    int a = 20;
    int b = 10;

    printf("First number: %d\n", a);
    printf("Second number: %d\n", b);

    printf("Addition: %d\n", add(a, b));
    printf("Subtraction: %d\n", subtract(a, b));

    return 0;
}
```

```
laasya@laasya: ~/module_lab × + v
laasya@laasya:~$ mkdir module_lab
laasya@laasya:~$ cd module_lab
laasya@laasya:~/module_lab$ nano calculator.h
laasya@laasya:~/module_lab$ nano calculator.c
laasya@laasya:~/module_lab$ nano main.c
laasya@laasya:~/module_lab$ gcc main.c calculator.c -o calculator
laasya@laasya:~/module_lab$ ./calculator
First number: 20
Second number: 10
Addition: 30
Subtraction: 10
laasya@laasya:~/module_lab$ |
```

TASK2:

To detect runtime errors, generate messages, log failures, handle invalid syntax, recover gracefully, and test error scenarios.

```

#include <stdio.h>
#include <stdlib.h>
#include <errno.h>
#include <string.h>

int main()
{
    int a, b;
    char operator;

    printf("Enter expression (example: 10 + 5): ");

    if (scanf("%d %c %d", &a, &operator, &b) != 3)
    {
        fprintf(stderr, "Error: Invalid input syntax.\n");
        return 1;
    }

    switch (operator)
    {
        case '+':
            printf("Result = %d\n", a + b);
            break;

        case '-':
            printf("Result = %d\n", a - b);
            break;

        case '*':
            printf("Result = %d\n", a * b);
            break;

        case '/':
            if (b == 0)
            {
                fprintf(stderr,
                    "Error: Division by zero is not allowed.\n");
            }
    }
}

```

^G Help
 ^X Exit

^O Write Out
 ^R Read File

^W Where Is
 ^\ Replace

^K Cut
 ^U Paste

^T Execute
 ^J Justify

```

        case '/':
            if (b == 0)
            {
                fprintf(stderr,
                    "Error: Division by zero is not allowed.\n");

                FILE *log = fopen("error.log", "a");

                if (log != NULL)
                {
                    fprintf(log,
                        "Division by zero attempted: %d / %d\n",
                        a, b);
                    fclose(log);
                }

                return 1;
            }

            printf("Result = %d\n", a / b);
            break;

        default:
            fprintf(stderr,
                "Error: Invalid operator '%c'.\n",
                operator);

            return 1;
    }

    return 0;
}

```

```

laasya@laasya:~/module_lab$ nano error_handling.c
laasya@laasya:~/module_lab$ gcc error_handling.c -o error_handling
laasya@laasya:~/module_lab$ ./error_handling
Enter expression (example: 10 + 5): 10+5
Result = 15
laasya@laasya:~/module_lab$ ./error_handling
Enter expression (example: 10 + 5): 10/0
Error: Division by zero is not allowed.
laasya@laasya:~/module_lab$ ./error_handling
Enter expression (example: 10 + 5): 10%5
Error: Invalid operator '%'.
laasya@laasya:~/module_lab$ ./error_handling
Enter expression (example: 10 + 5): hello
Error: Invalid input syntax.
laasya@laasya:~/module_lab$ |

```

TASK3:

To run Valgrind, identify leaks, fix memory errors, release resources, verify cleanup, and document findings.

```
#include <stdio.h>
#include <stdlib.h>

int main()
{
    int *numbers;

    numbers = malloc(5 * sizeof(int));

    if (numbers == NULL)
    {
        printf("Memory allocation failed.\n");
        return 1;
    }

    for (int i = 0; i < 5; i++)
    {
        numbers[i] = i + 1;
    }

    printf("Numbers:\n");

    for (int i = 0; i < 5; i++)
    {
        printf("%d ", numbers[i]);
    }

    printf("\n");

    /*
     * Intentionally not freeing memory.
     */

    return 0;
}
```

```

1 2 3 4 5
==989==
==989== HEAP SUMMARY:
==989==   in use at exit: 20 bytes in 1 blocks
==989==   total heap usage: 2 allocs, 1 frees, 1,044 bytes allocated
==989==
==989== 20 bytes in 1 blocks are definitely lost in loss record 1 of 1
==989==   at 0x4846828: malloc (in /usr/libexec/valgrind/vgpreload_memcheck
-amd64-linux.so)
==989==   by 0x1091BE: main (memory_test.c:8)
==989==
==989== LEAK SUMMARY:
==989==   definitely lost: 20 bytes in 1 blocks
==989==   indirectly lost: 0 bytes in 0 blocks
==989==   possibly lost: 0 bytes in 0 blocks
==989==   still reachable: 0 bytes in 0 blocks
==989==   suppressed: 0 bytes in 0 blocks
==989==
==989== For lists of detected and suppressed errors, rerun with: -s
==989== ERROR SUMMARY: 1 errors from 1 contexts (suppressed: 0 from 0)
laasya@laasya:~/module_lab$ nano memory_test.c
laasya@laasya:~/module_lab$ gcc -g memory_test.c -o memory_test
laasya@laasya:~/module_lab$ valgrind --leak-check=full ./memory_test
==1000== Memcheck, a memory error detector
==1000== Copyright (C) 2002-2022, and GNU GPL'd, by Julian Seward et al.
==1000== Using Valgrind-3.22.0 and LibVEX; rerun with -h for copyright info
==1000== Command: ./memory_test
==1000==
Numbers:
1 2 3 4 5
==1000==
==1000== HEAP SUMMARY:
==1000==   in use at exit: 0 bytes in 0 blocks
==1000==   total heap usage: 2 allocs, 2 frees, 1,044 bytes allocated
==1000==
==1000== All heap blocks were freed -- no leaks are possible
==1000==
==1000== For lists of detected and suppressed errors, rerun with: -s
==1000== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)
laasya@laasya:~/module_lab$ |

```

TASK 4:

To design test cases, create test scripts, automate execution, verify outputs, compare results, and generate reports.

```
#!/bin/bash

echo "====="
echo "      CALCULATOR TESTS"
echo "====="

echo ""
echo "Test 1: Addition"

echo "10 + 5" | ./error_handling > output1.txt

if grep -q "Result = 15" output1.txt
then
    echo "PASS"
else
    echo "FAIL"
fi

echo ""
echo "Test 2: Subtraction"

echo "10 - 5" | ./error_handling > output2.txt

if grep -q "Result = 5" output2.txt
then
    echo "PASS"
else
    echo "FAIL"
fi

echo ""
echo "Test 3: Multiplication"

echo "10 * 5" | ./error_handling > output3.txt
```

```
        echo "PASS"
    else
        echo "FAIL"
    fi

    echo ""
    echo "Test 4: Division"

    echo "10 / 5" | ./error_handling > output4.txt

    if grep -q "Result = 2" output4.txt
    then
        echo "PASS"
    else
        echo "FAIL"
    fi

    echo ""
    echo "Test 5: Division by zero"

    echo "10 / 0" | ./error_handling > output5.txt 2>&1

    if grep -q "Division by zero" output5.txt
    then
        echo "PASS"
    else
        echo "FAIL"
    fi

    echo ""
    echo "=====
    echo "          TESTING COMPLETE"
    echo "=====
|
```

```
laasya@laasya:~/module_lab$ nano test.sh
laasya@laasya:~/module_lab$ chmod +x test.sh
laasya@laasya:~/module_lab$ ./test.sh
=====
          CALCULATOR TESTS
=====

Test 1: Addition
PASS

Test 2: Subtraction
PASS

Test 3: Multiplication
PASS

Test 4: Division
PASS

Test 5: Division by zero
PASS

=====
          TESTING COMPLETE
=====
laasya@laasya:~/module_lab$ ls
calculator      error.log      main.c         output1.txt    output4.txt
calculator.c    error_handling memory_test    output2.txt    output5.txt
calculator.h    error_handling.c memory_test.c  output3.txt    test.sh
laasya@laasya:~/module_lab$ |
```